DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CSE 3102: Software Engineering Sessional

FarmLink

Software Requirements Specification, ER Diagram & System Design

Phase 1 Deliverable — Requirement Analysis, SRS Documentation, ER Diagram & System Design

Submitted By	Submitted To
Name: Md. Asif Hasan ID: 0802420205101032 Name: Motmainna Akter ID: 0802420105101031 Name: Tanbin Islam Krity ID: 0802420105101033	Name: Md. Rakib Trofder Designation: Lecturer, CSE Name: Md. Sirajul Islam Designation: Lecturer, CSE

Table of Contents

Table of Contents 2
1. Introduction 3
1.1 Purpose 3
1.2 Scope 3
1.3 Intended Audience 3
1.4 Definitions, Acronyms & Abbreviations 3
2. Overall Description 4
2.1 Product Perspective 4
2.2 User Classes and Characteristics 4
2.3 Operating Environment 4
2.4 Assumptions and Dependencies 4
3. Functional Requirements 5
FR-1: User Management & Authentication 5
FR-2: Surplus Food Posting 5
FR-3: Smart Matching & Notification 5
FR-4: Animal Health & Nutrition Validation 6
FR-5: Pickup Verification 6
FR-6: Transaction & Reward Management 6
FR-7: Analytics & Reporting 6
FR-8: Notification & Communication 6
4. Non-Functional Requirements 8
5. External Interface Requirements 8
6. System Use Cases 9
6.1 Use Case Summary 9
6.2 Detailed Use Case — Post Surplus Food 9
6.3 Detailed Use Case — Scan QR at Pickup 10
7. Entity-Relationship (ER) Diagram 11
7.1 Entity / Data Dictionary 12
8. System Design Overview 13
8.1 Architectural Style 13
8.2 Module Decomposition 13
8.3 Design Rationale 13
9. Conclusion 13

1. Introduction

1.1 Purpose

This document specifies the software requirements, entity-relationship model, and system design for FarmLink — a mobile-based platform that connects university halls with local livestock farms to redistribute surplus food. It serves as the Phase 1 deliverable of the CSE 3102 Software Engineering Sessional project, guiding subsequent design and implementation phases.

1.2 Scope

FarmLink covers three connected mobile/web experiences — a Hall Admin app, a Farm Owner app, and a Super Admin web dashboard — backed by a shared cloud database. The scope includes surplus food posting, automated farm matching, animal-health-aware validation, QR-based pickup verification, exchange/transaction tracking, and analytics reporting. Payment-gateway integration is treated as an optional, later-phase extension.

1.3 Intended Audience

This document is intended for the course instructor evaluating the project, the development team implementing the system, and any future contributors extending FarmLink beyond the sessional scope.

1.4 Definitions, Acronyms & Abbreviations

Term	Definition
SRS	Software Requirements Specification
ERD	Entity-Relationship Diagram
FCM	Firebase Cloud Messaging
RLS	Row Level Security (Supabase/PostgreSQL access control)
QR Code	Quick Response code used for pickup verification
Hall Admin	Staff member who manages surplus food posting on behalf of a university hall
Farm Owner	Registered livestock farm operator who collects surplus food
Super Admin	System-level administrator overseeing all halls and farms

2. Overall Description

2.1 Product Perspective

FarmLink is a new, self-contained cross-platform system. It is not a replacement for any existing institutional software, but introduces a new digital workflow that formalizes what is currently an ad hoc, manual process between halls and farms.

2.2 User Classes and Characteristics

User Class	Description	Technical Expertise
Hall Admin	Posts surplus food details, tracks exchange history, views hall-level reports.	Low to Moderate
Farm Owner	Receives match notifications, manages animal profiles, confirms pickups.	Low
Super Admin	Manages user/farm approvals, monitors system-wide analytics, configures nutrition guidelines.	Moderate to High

2.3 Operating Environment

The mobile application runs on Android 8.0+ and iOS 13+ smartphones. The backend operates on Firebase or Supabase cloud infrastructure. The Super Admin dashboard runs as a responsive web application accessible via any modern browser.

2.4 Assumptions and Dependencies

- Hall Admins and Farm Owners have access to a smartphone with an active internet connection (with SMS fallback for low-connectivity cases).
- Farms participating in the system are willing to register verifiable animal and location details.
- Third-party services (Firebase/Supabase, Google Maps, Twilio) remain available and within free/paid tier limits appropriate for the project scale.

3. Functional Requirements

Functional requirements are grouped by system module below, each identified with a unique ID and priority level.

FR-1: User Management & Authentication

ID	Requirement	Priority
FR-1.1	The system shall allow new users to register with role selection (Hall Admin or Farm Owner).	High
FR-1.2	The system shall authenticate users via secure email/password login (Firebase/Supabase Auth).	High
FR-1.3	The system shall enforce role-based access so users only reach features/data permitted for their role.	High
FR-1.4	The system shall allow users to reset a forgotten password via email/OTP verification.	Medium

FR-2: Surplus Food Posting

ID	Requirement	Priority
FR-2.1	Hall Admin shall be able to create a food post specifying food type, quantity, and expected pickup time.	High
FR-2.2	The system shall allow editing or cancelling a post before it is matched to a farm.	Medium
FR-2.3	The system shall timestamp and track each post's status (Open, Matched, Collected, Expired).	High

FR-3: Smart Matching & Notification

ID	Requirement	Priority
FR-3.1	The system shall identify eligible registered farms automatically based on location and animal-type compatibility.	High
FR-3.2	The system shall send a real-time push notification (FCM) to matched farm owners for each new suitable post.	High
FR-3.3	Farm Owner shall be able to accept or decline a match request from within the app.	High
FR-3.4	The system shall send an SMS fallback notification (Twilio) when push delivery fails or connectivity is limited.	Medium
FR-3.5	If a match is declined or expires, the system shall automatically reassign the post to the next eligible farm.	Medium

FR-4: Animal Health & Nutrition Validation

ID	Requirement	Priority
FR-4.1	The system shall maintain a nutrition guideline dataset mapping animal species to safe food types and quantities.	High
FR-4.2	The system shall validate each match against nutrition guidelines and warn if a food type is unsuitable.	High
FR-4.3	Farm Owner shall be able to register and update animal profiles (species, count, health status).	Medium

FR-5: Pickup Verification

ID	Requirement	Priority
FR-5.1	The system shall generate a unique QR code for every confirmed match.	High
FR-5.2	Farm Owner shall scan the QR code at pickup to confirm collection.	High
FR-5.3	The system shall log pickup time, quantity collected, and the verifying user.	Medium

FR-6: Transaction & Reward Management

ID	Requirement	Priority
FR-6.1	The system shall record the return exchange (milk, eggs, meat, or cash) for each completed match.	High
FR-6.2	The system shall maintain a transaction history per hall and per farm.	Medium
FR-6.3	The system shall calculate cumulative exchange summaries (e.g., monthly totals).	Medium

FR-7: Analytics & Reporting

ID	Requirement	Priority
FR-7.1	The system shall provide a dashboard showing total food redistributed, active farms, and waste-reduction metrics.	Medium
FR-7.2	The system shall generate visual charts (bar/line/pie) for hall-wise and farm-wise activity.	Medium
FR-7.3	Super Admin shall be able to export reports for institutional use.	Low

FR-8: Notification & Communication

ID	Requirement	Priority
FR-8.1	The system shall provide in-app notifications for match requests, confirmations, and reminders.	High

FR-8.2	The system shall allow basic in-app messaging between Hall Admin and Farm Owner for coordination.	Low

4. Non-Functional Requirements

Category	Requirement
Performance	The system shall dispatch match notifications within 5 seconds of a new surplus food post.
Scalability	The cloud backend (Firebase/Supabase) shall support a growing number of halls and farms without architectural redesign.
Security	All user data shall be protected via role-based access control, encrypted authentication, and Firebase Security Rules / Supabase RLS.
Usability	The mobile interface shall be simple and icon-driven to remain usable by farm owners with limited technical literacy, with Bengali and English language support.
Reliability	The system shall maintain at least 99% uptime and support offline local caching during network interruptions.
Maintainability	The codebase shall follow modular, layered architecture (Riverpod-based state management) to simplify future updates.
Portability	The mobile application shall run on both Android and iOS from a single Flutter codebase.
Availability	The system shall be accessible 24/7 through cloud hosting with automatic backups.

5. External Interface Requirements

Interface Type	Details
User Interface	Flutter mobile app (Hall Admin & Farm Owner views) and a web-based dashboard (Super Admin view).
Hardware Interface	Smartphone camera (QR code scanning), GPS module (farm/hall location).
Software Interface	Firebase/Supabase SDK, Google Maps API, Twilio SMS API, FCM.
Communication Interface	HTTPS REST APIs, real-time database listeners, push-notification protocol (FCM).

6. System Use Cases

6.1 Use Case Summary

ID	Use Case	Primary Actor	Description
UC-01	Register / Login	All Users	Create an account and authenticate using role-based credentials.
UC-02	Post Surplus Food	Hall Admin	Create a food post specifying type, quantity, and pickup time.
UC-03	Receive Match Notification	Farm Owner	Get notified when a compatible surplus food post is available.
UC-04	Accept / Decline Match	Farm Owner	Respond to a matched food-post request.
UC-05	Manage Animal Profile	Farm Owner	Add or update registered animal species, count, and health status.
UC-06	Validate Nutrition Compatibility	System	Automatically check posted food against nutrition guidelines before confirming a match.
UC-07	Scan QR at Pickup	Farm Owner	Confirm physical collection of food using QR verification.
UC-08	Record Exchange / Transaction	Hall Admin, Farm Owner	Log the return goods (milk/eggs/meat) or cash exchanged.
UC-09	View Analytics Dashboard	Super Admin, Hall Admin	Monitor food-waste reduction and exchange statistics.
UC-10	Manage Users & Farms	Super Admin	Approve, suspend, or manage registered hall and farm accounts.

6.2 Detailed Use Case — Post Surplus Food

Field	Description
Use Case ID	UC-02
Actor	Hall Admin
Precondition	Hall Admin is authenticated and has an active hall profile.
Main Flow	1. Hall Admin opens “Post Surplus Food” screen. 2. Enters food type, quantity, and expected pickup time. 3. Submits the post. 4. System validates input, stores the post with status “Open”, and triggers the matching engine.
Postcondition	A new FoodPost record is created and eligible farms are notified.
Alternate Flow	If required fields are missing, the system displays a validation error and the post is not submitted.

6.3 Detailed Use Case — Scan QR at Pickup

Field	Description
Use Case ID	UC-07
Actor	Farm Owner
Precondition	A match has been accepted and a QR code has been generated for pickup.
Main Flow	1. Farm Owner arrives at the hall and opens the “Scan Pickup” screen. 2. Scans the QR code linked to the match. 3. System verifies the code and timestamps the pickup. 4. Match status updates to “Collected” and a transaction record is initiated.
Postcondition	A Pickup record is logged, and the associated Transaction is ready for exchange details.
Alternate Flow	If the QR code is invalid or already used, the system rejects the scan and displays an error.

7. Entity-Relationship (ER) Diagram

The diagram below models the core data entities of FarmLink and their relationships, forming the basis for the Firestore/PostgreSQL schema design.

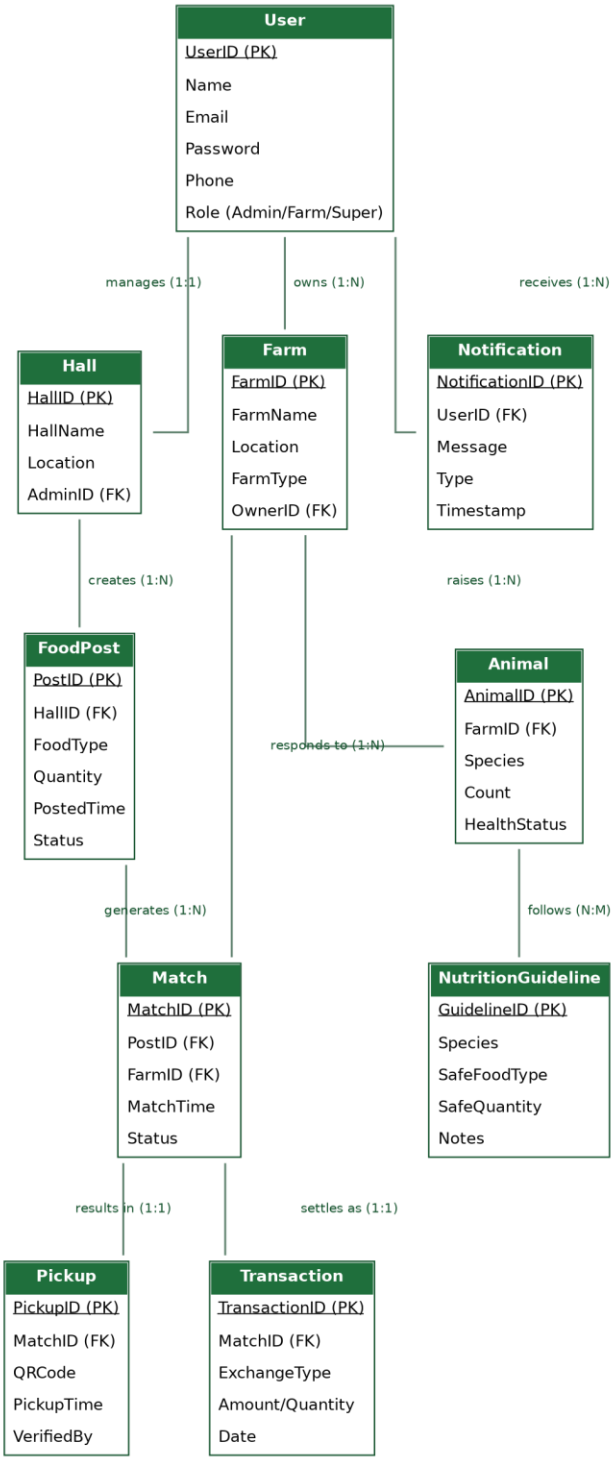


Figure 1: Entity-Relationship Diagram of the FarmLink System

7.1 Entity / Data Dictionary

Entity	Key Attributes	Description
User	UserID (PK), Name, Email, Password, Phone, Role	Base account for Hall Admin, Farm Owner, and Super Admin, distinguished by Role.
Hall	HallID (PK), HallName, Location, AdminID (FK)	Represents a university residential hall managed by a Hall Admin.
Farm	FarmID (PK), FarmName, Location, FarmType, OwnerID (FK)	Represents a registered livestock farm owned by a Farm Owner.
Animal	AnimalID (PK), FarmID (FK), Species, Count, HealthStatus	Livestock records maintained by each farm for nutrition matching.
FoodPost	PostID (PK), HallID (FK), FoodType, Quantity, PostedTime, Status	A surplus food listing created by a Hall Admin.
Match	MatchID (PK), PostID (FK), FarmID (FK), MatchTime, Status	Links a FoodPost to a responding Farm.
Pickup	PickupID (PK), MatchID (FK), QRCode, PickupTime, VerifiedBy	Records QR-verified collection details for a match.
Transaction	TransactionID (PK), MatchID (FK), ExchangeType, Amount/Quantity, Date	Records the return exchange (milk/eggs/meat/cash) for a completed match.
NutritionGuideline	GuidelineID (PK), Species, SafeFoodType, SafeQuantity, Notes	Reference data used to validate food safety per animal species.
Notification	NotificationID (PK), UserID (FK), Message, Type, Timestamp	In-app/push/SMS notification log delivered to a user.

8. System Design Overview

8.1 Architectural Style

FarmLink follows a three-tier architecture. The Presentation Layer consists of the Flutter mobile apps and the web admin dashboard. The Application/Logic Layer is implemented through Firebase Cloud Functions or Supabase Edge Functions, handling matching logic, nutrition validation, and transaction calculations. The Data Layer stores all persistent records in Firestore or PostgreSQL, secured through role-based access rules.

8.2 Module Decomposition

The backend logic is organized into the following independent modules, each mapped to the functional requirement groups in Section 3:

- auth_module — registration, login, role-based access control
- food_post_module — create/edit/track surplus food posts
- matching_module — eligibility filtering, auto-matching, reassignment logic
- nutrition_module — animal profiles, nutrition guideline validation
- pickup_module — QR generation, scanning, pickup logging
- transaction_module — exchange recording, history, summaries
- notification_module — FCM push notifications, Twilio SMS fallback
- analytics_module — dashboard metrics, charts, report export
- admin_module — user/farm approval, guideline management (Super Admin)

8.3 Design Rationale

Separating concerns into distinct modules keeps the matching engine, nutrition validation, and transaction logic independently testable, and allows the team to divide implementation work by module during the sessional timeline without tight coupling between components.

9. Conclusion

This document establishes the functional and non-functional requirements, use cases, entity-relationship model, and high-level system design for FarmLink. It provides the foundation for the subsequent design and implementation phases outlined in the project work plan, ensuring development proceeds against a clearly defined and traceable set of requirements.